Design

Chapter 9

Introduction

- Design is the start of building the system
- We shape the system and find its form
  - including architectural core
- Design model builds from analysis model
- Design changes the focus towards implementation

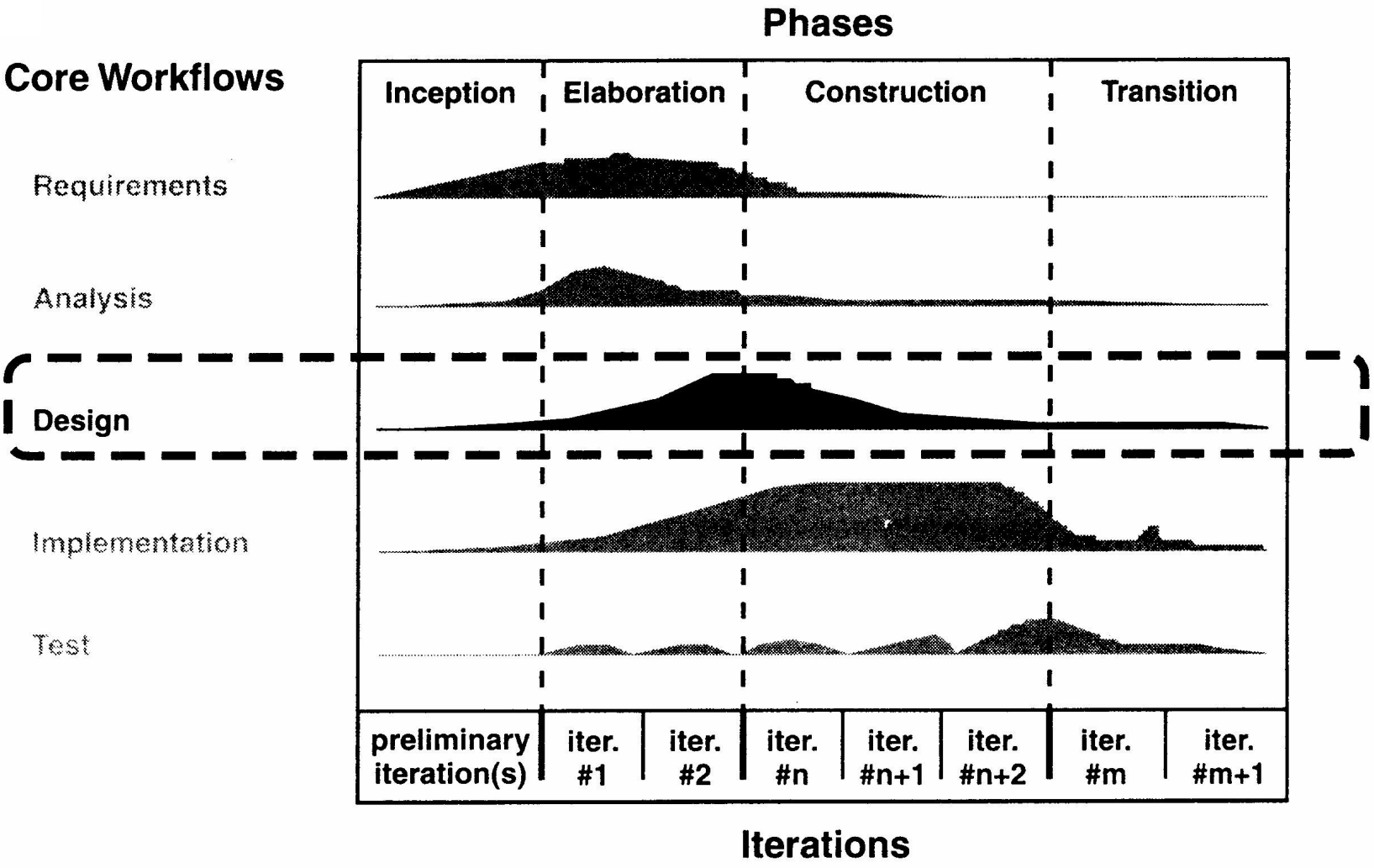
Purposes of Design

- Understand nonfunctional issues related to
  - programming languages
  - component reuse
  - operating systems
  - distribution and concurrency technologies
  - database technologies
  - user-interface technologies
  - transaction management technologies

Purposes of Design

- Create input for implementation
  - specify subsystems, interfaces, classes
- Decompose implementation into manageable pieces
  - often for implementation by different teams
- Capture and control major interfaces early
  - critical for gaining control over implementation
- Create seamless abstraction
  - structure preserved into code

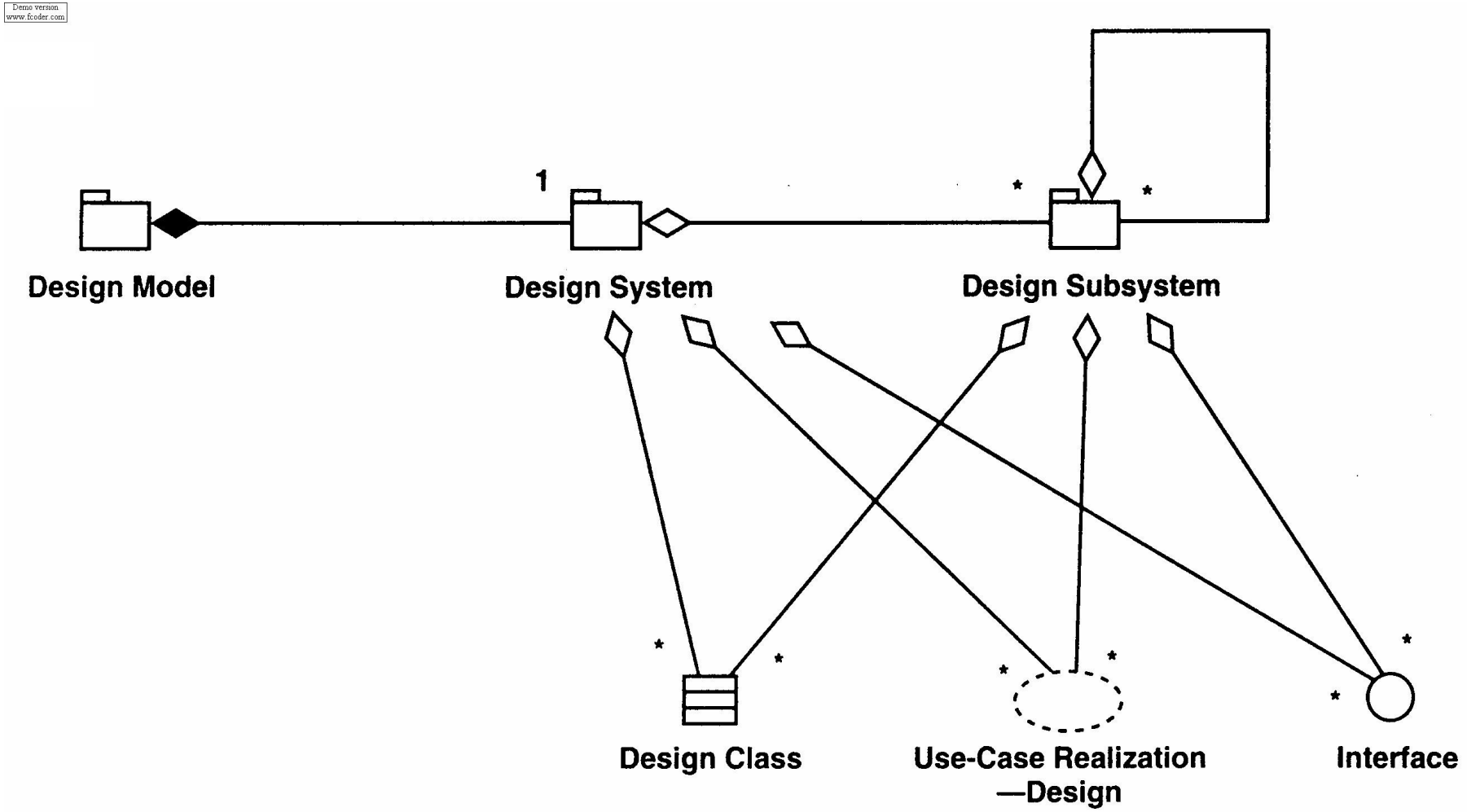
Place of Design in Software Life Cycle



Artifacts

- Design Model
- Design Class
- Use-Case realization - design
- Design subsystem
- Interface
- Architecture description (view of design model)
- Deployment model
- Architecture description (view of deployment model)

Design Model



Analysis vs. Design Model

- Conceptual model
- Design independent
- Analysis classes
- Less formal
- Less expensive
- Few layers
- Little focus on sequence
- Outlines design
- May not be maintained

- Physical model
- Implementation specific
- Physical stereotypes
- More formal
- 5:1 more expensive
- Many layers
- Much focus on sequence
- Manifests design
- Should be maintained

Further Differences: Analysis vs. Design Models

- Created by hand
- Defines structure that shapes the system
- May be created by visual programming and round-trip engineering
- Preserves analysis model structure as much as possible

Design Class

- Define
  - operations
  - attributes
  - relationships
  - methods
  - implementation requirements
  - whether active or not
- May realize an interface

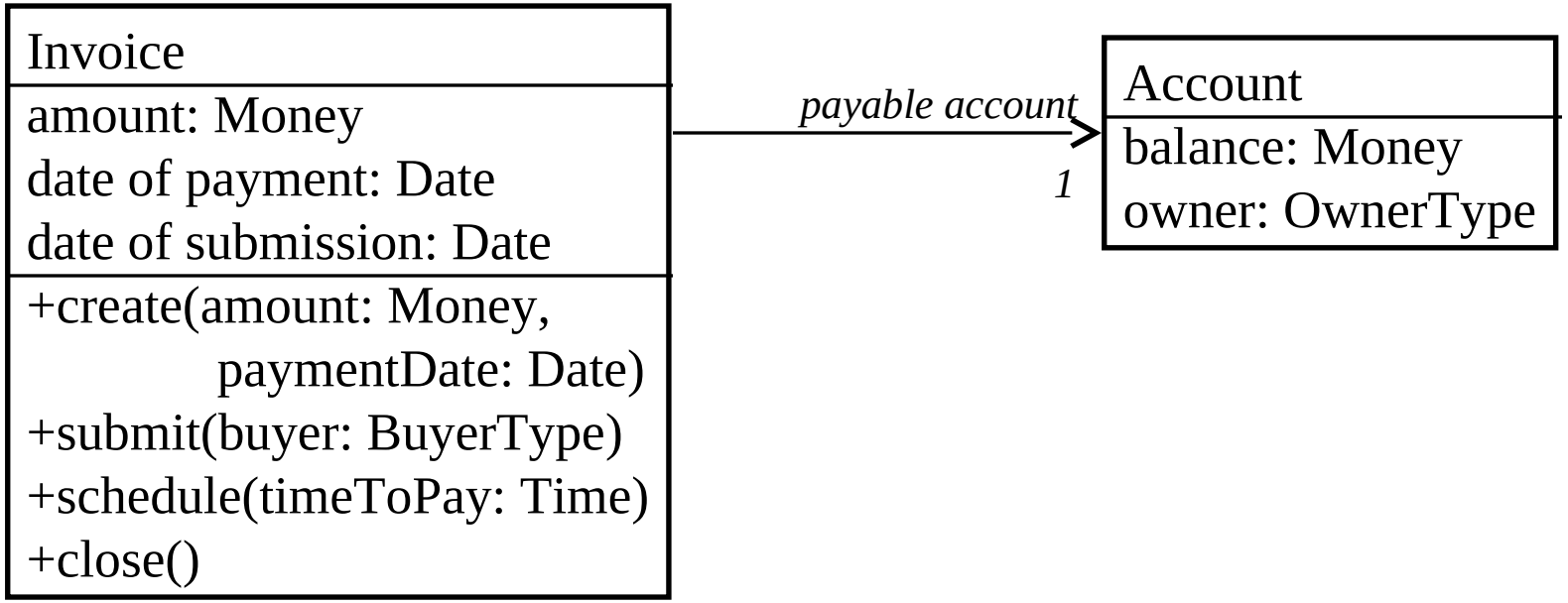
Design Classes Seamless to Implementation

- Language used to specify design class taken from implementation language
- Visibility specified in language-dependent terms
  - public, protected, private for C++
- Design relationships map to implementation ones
  - generalization to inheritance
  - associations to references
- methods map to implementation methods

Design Classes Seamless to Implementation

- Design classes may be given stereotypes to indicate programming language constructs
  - C# «class module», «form», «user control»
- If implementation language has interfaces, class can realize it
- Design classes can be active
  - objects of class maintain their own thread of control
  - run concurrently with other active objects
  - depends on the programming language used
  - may depend on operating system used

Example:
Invoice Design Class



Concurrency

- Concurrency: "active" & "passive" objects
- Object interface techniques are the same whether there is concurrency or not
  - in analysis level *all* objects are potentially concurrent
- When coded, some objects contain 'tasks' and others are sequential
  - if all objects were concurrent, performance suffers
- Booch classified objects to find concurrency
  - actors, agents and servers.

Actors, Agents and Servers

- According to Booch, objects can be categorized
  - whether they perform an active or passive role
- ACTOR: not acted upon by other objects
  - usually acts on other objects
- SERVER: is acted upon by other objects
  - doesn't act upon anything else
- AGENT: performs some operation on behalf of another object
  - may act upon other objects to do its job

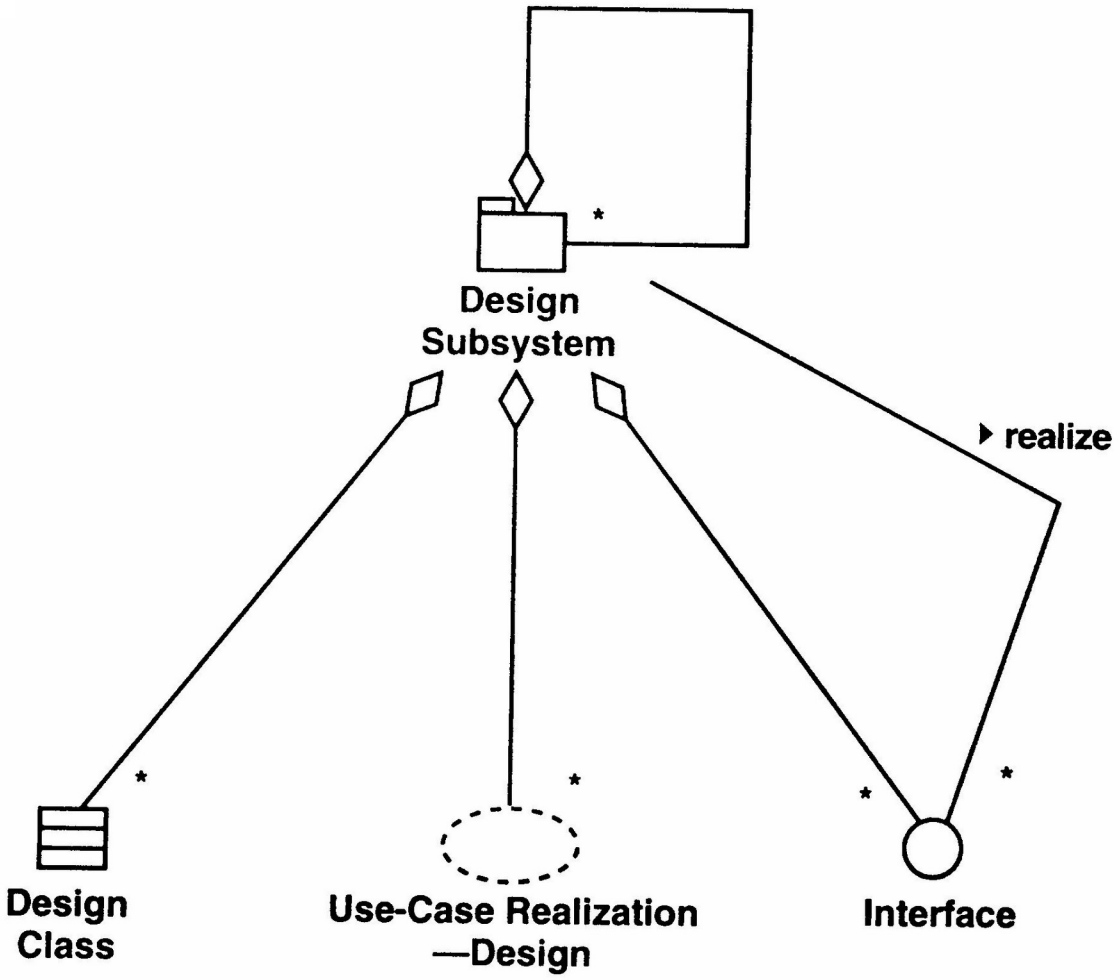
Use-Case Realization - Design

- Contains
  - class diagrams
    - can use to show participating classes
  - interaction diagrams
    - sequence diagrams show this graphically
  - flow of Events - Design
    - textual description
  - implementation requirements
    - textual description

Design Subsystem

- Provides means of organizing design artifacts into more manageable pieces
- Contains design classes, use-case realizations, interfaces, other subsystems
- Represents hierarchical decomposition of the system
- Can provide interfaces to export their functionality

Subsystem Contents



Subsystem Characteristics

- Should be cohesive
- Should be loosely coupled to other subsystems
- May separate system into parts to be developed by different teams
- Help to describe layers in system architecture
- Represent large-scale analogues to components
  - complete with interfaces, or APIs
- Help support reuse

Interface

- Specifies operations provided by design classes and subsystems
- Interfaces realized by design class methods or subsystem class methods
- Subsystem interfaces are architecturally significant
- Stable interfaces allow developers to work independently to implement subsystems

Architecture Description - View of Design Model

- Contains architectural view of design model
- Following usually architecturally significant:
  - subsystems, their interfaces and dependencies
  - key design classes
    - classes that trace to key analysis classes
    - active classes
    - classes that are general and central
  - use case realizations that have important and critical functionality

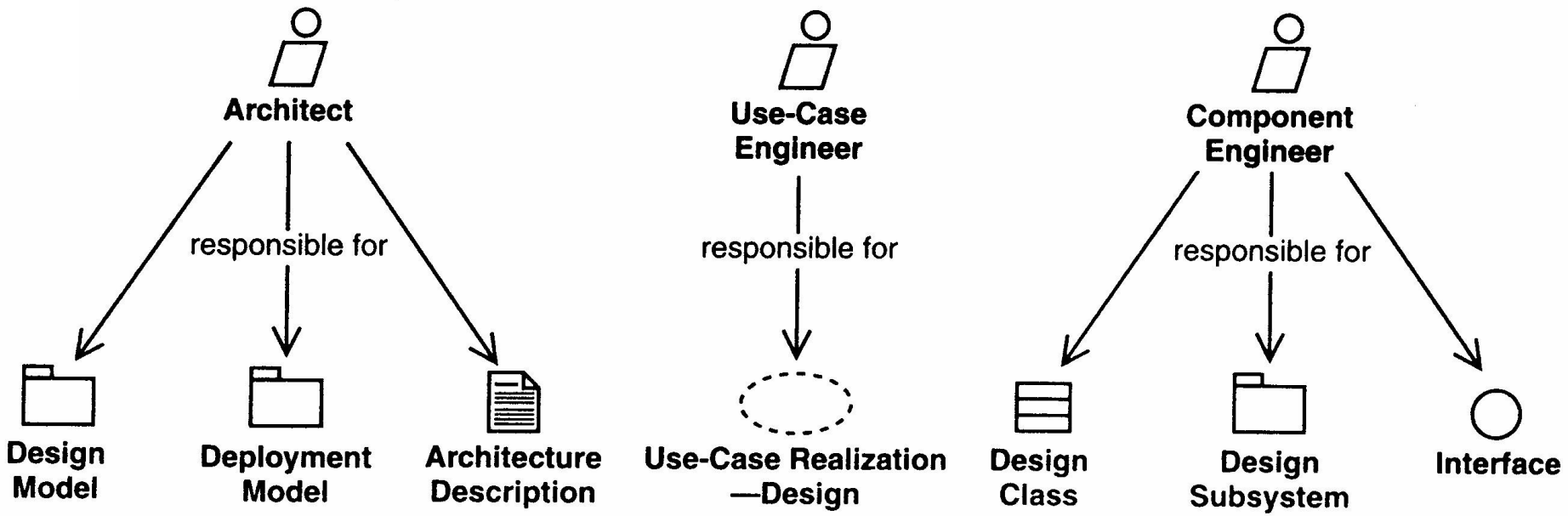
Deployment Model

- Physical distribution of system among computational nodes
- Nodes often represents processors
- Nodes have relationships that indicate communications mechanism
- Can have test and simulation versions
- Components deployed on a node define its functionality

Architecture Description - View of Deployment Model

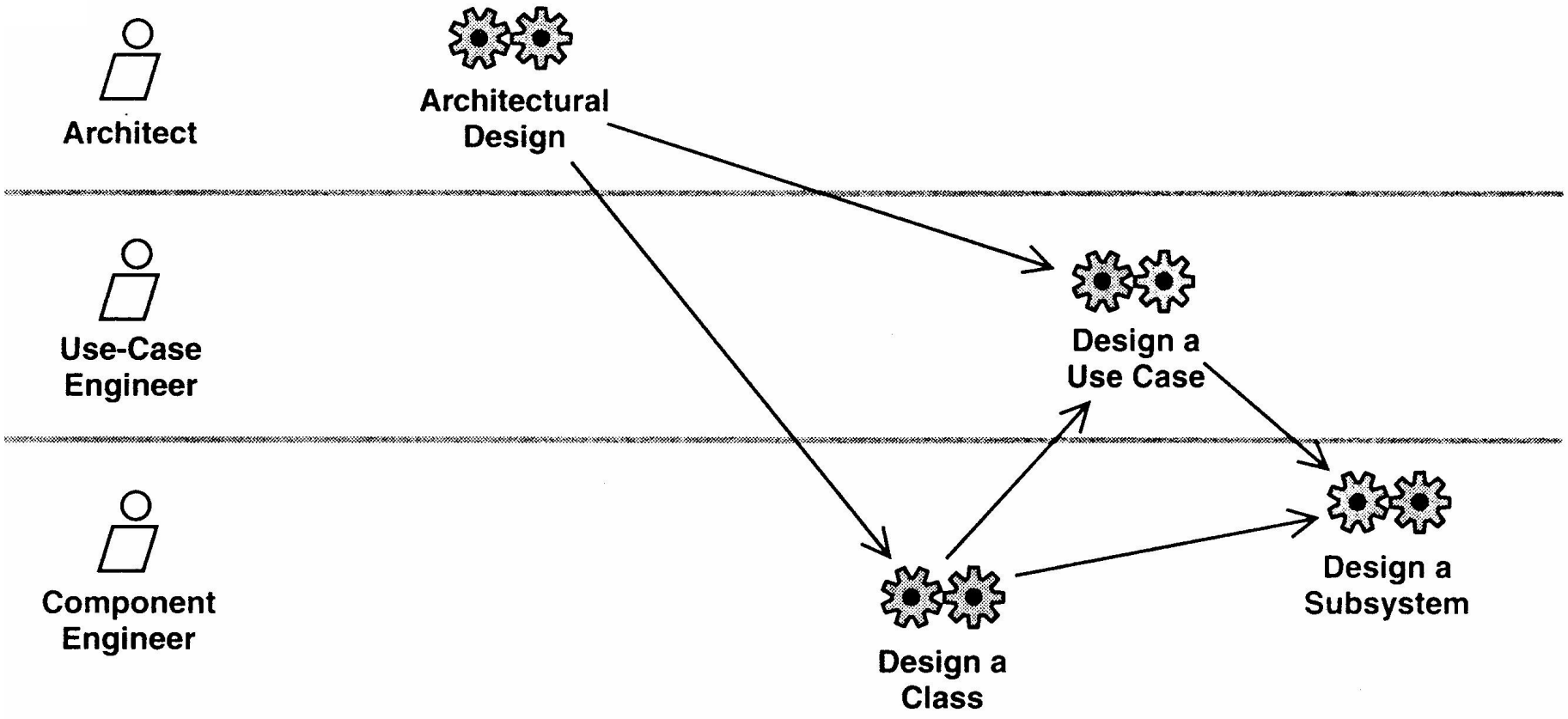
- Usually the entire deployment model is part of the architectural view of the deployment model

Roles



Architecture description is a view of the design model and deployment model

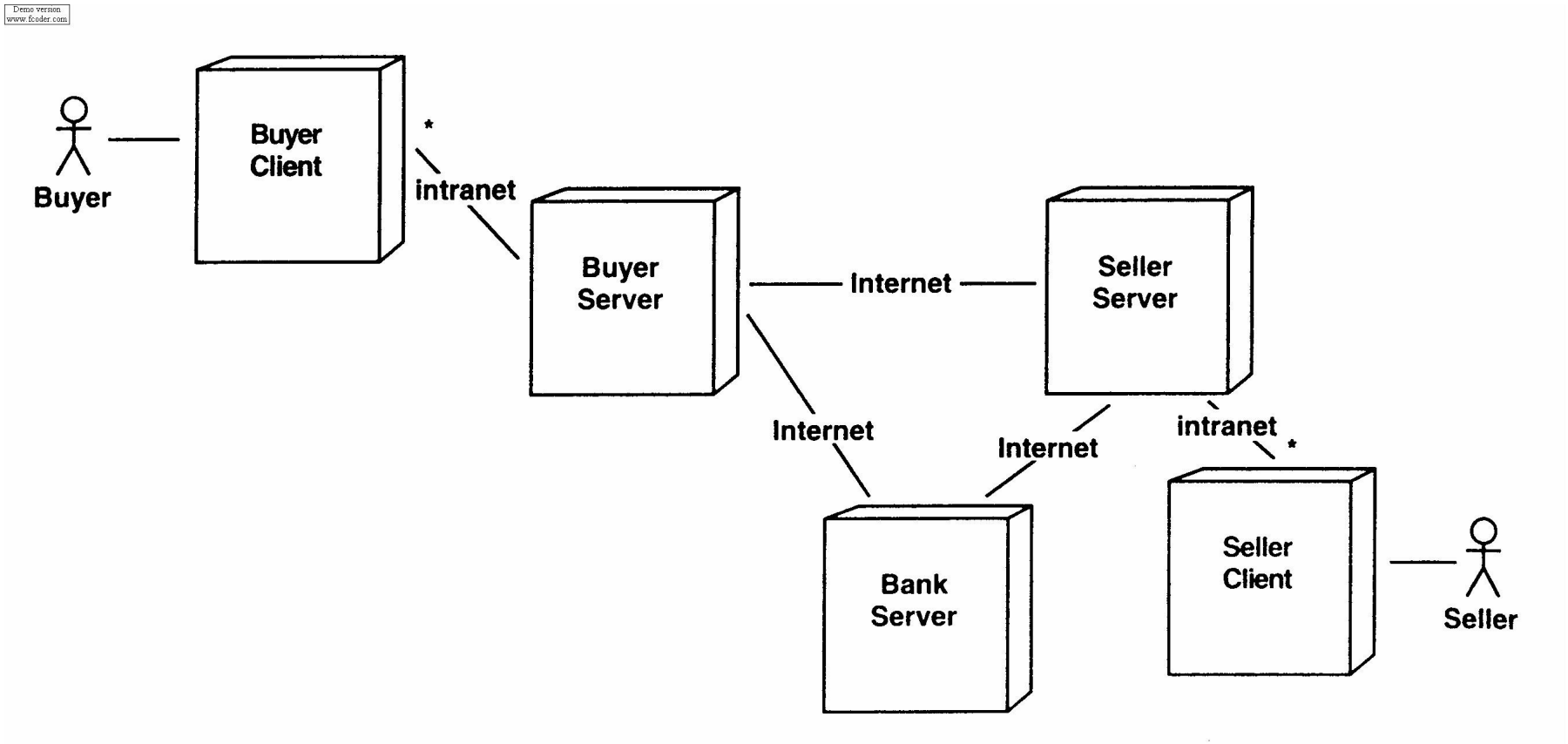
Workflow


Architect
Architectural Design
Use-Case Engineer
Design a Use Case
Component Engineer
Design a Class
Design a Subsystem

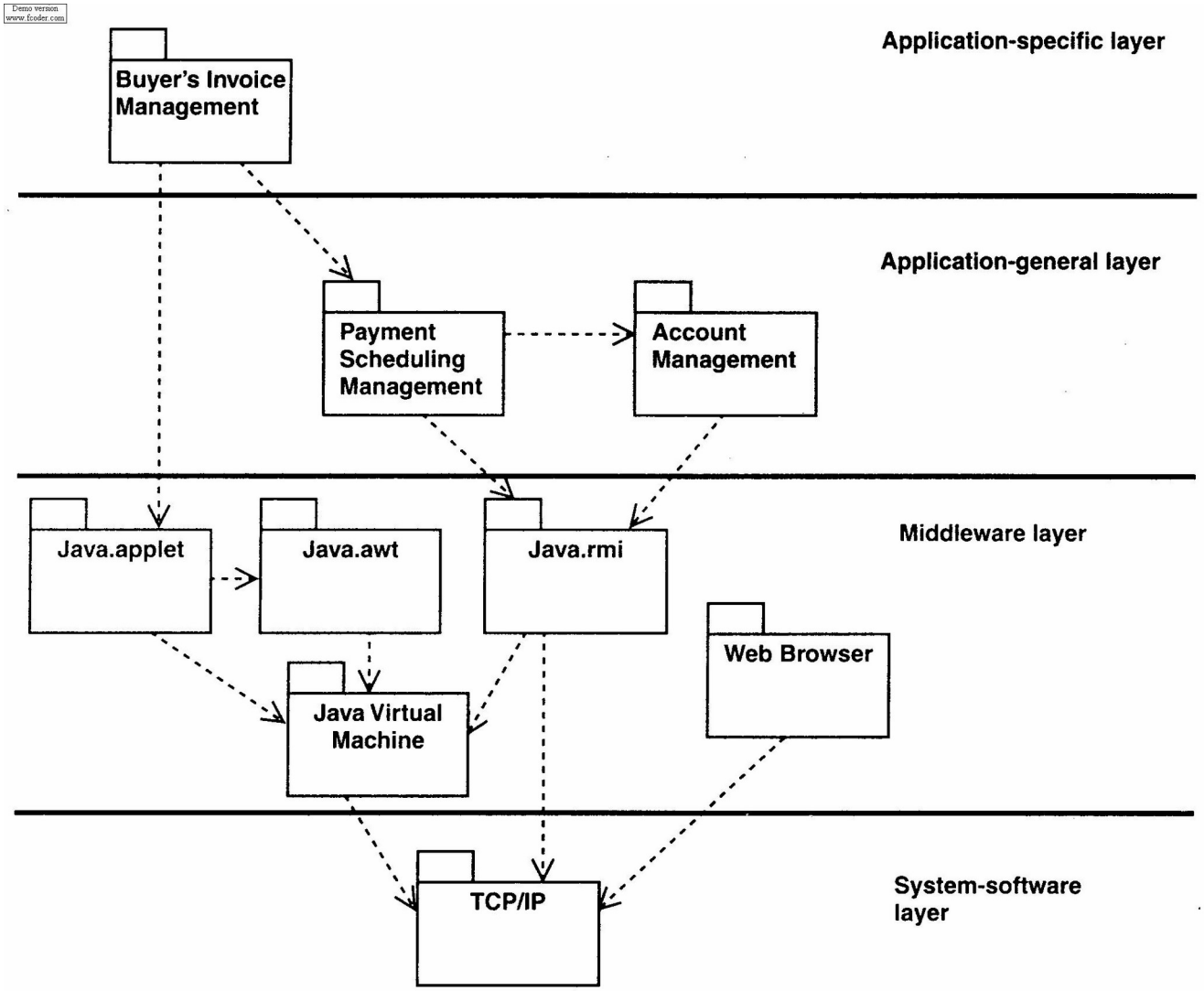
Architectural Design

- Identify nodes and network configurations
- Identify subsystems and interfaces
- Identify architecturally significant design classes
- Identify generic design mechanisms
  - persistency
  - transparent object distribution
  - security features
  - error detection and recovery
  - transaction management

Network Configuration



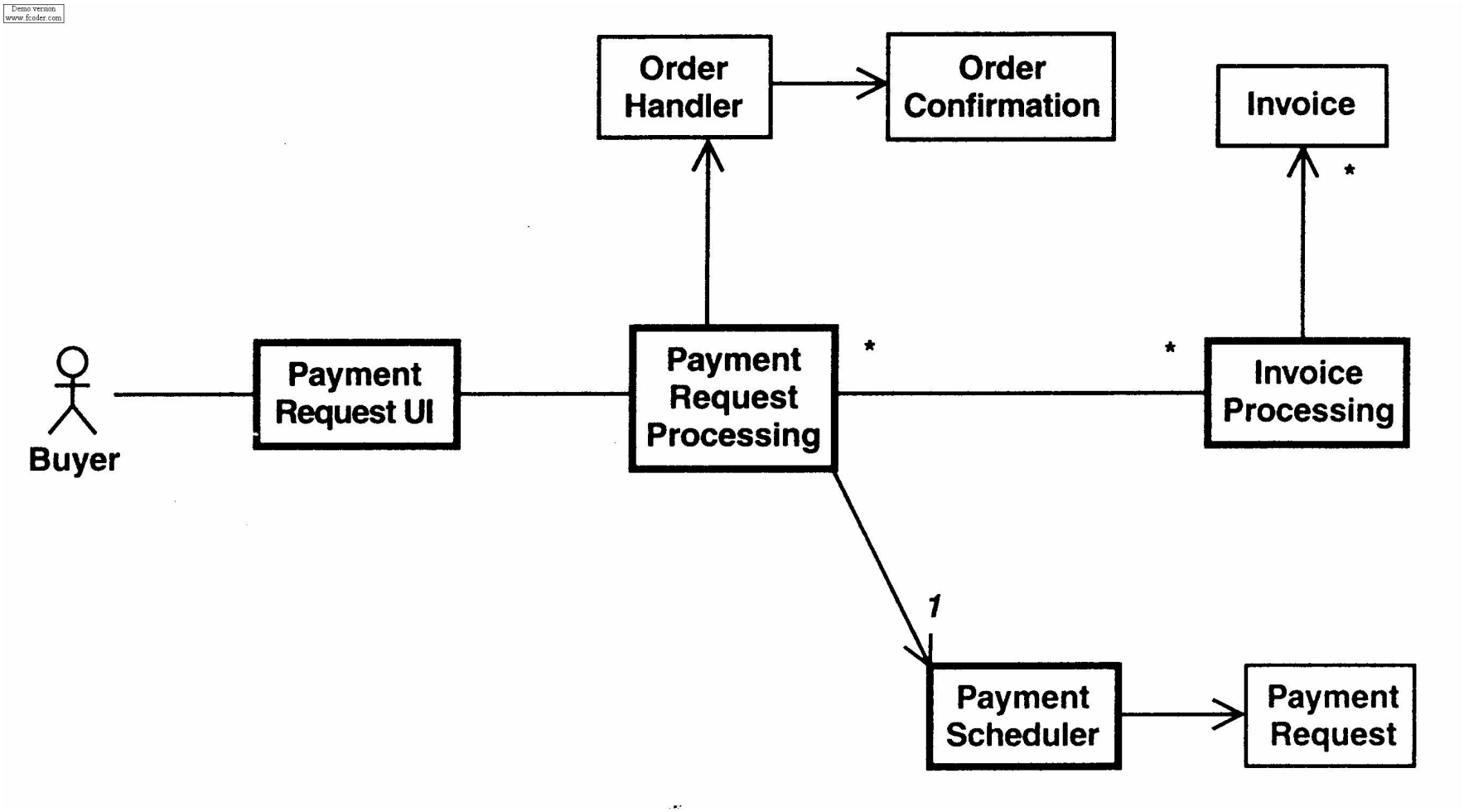
Subsystems and Interfaces



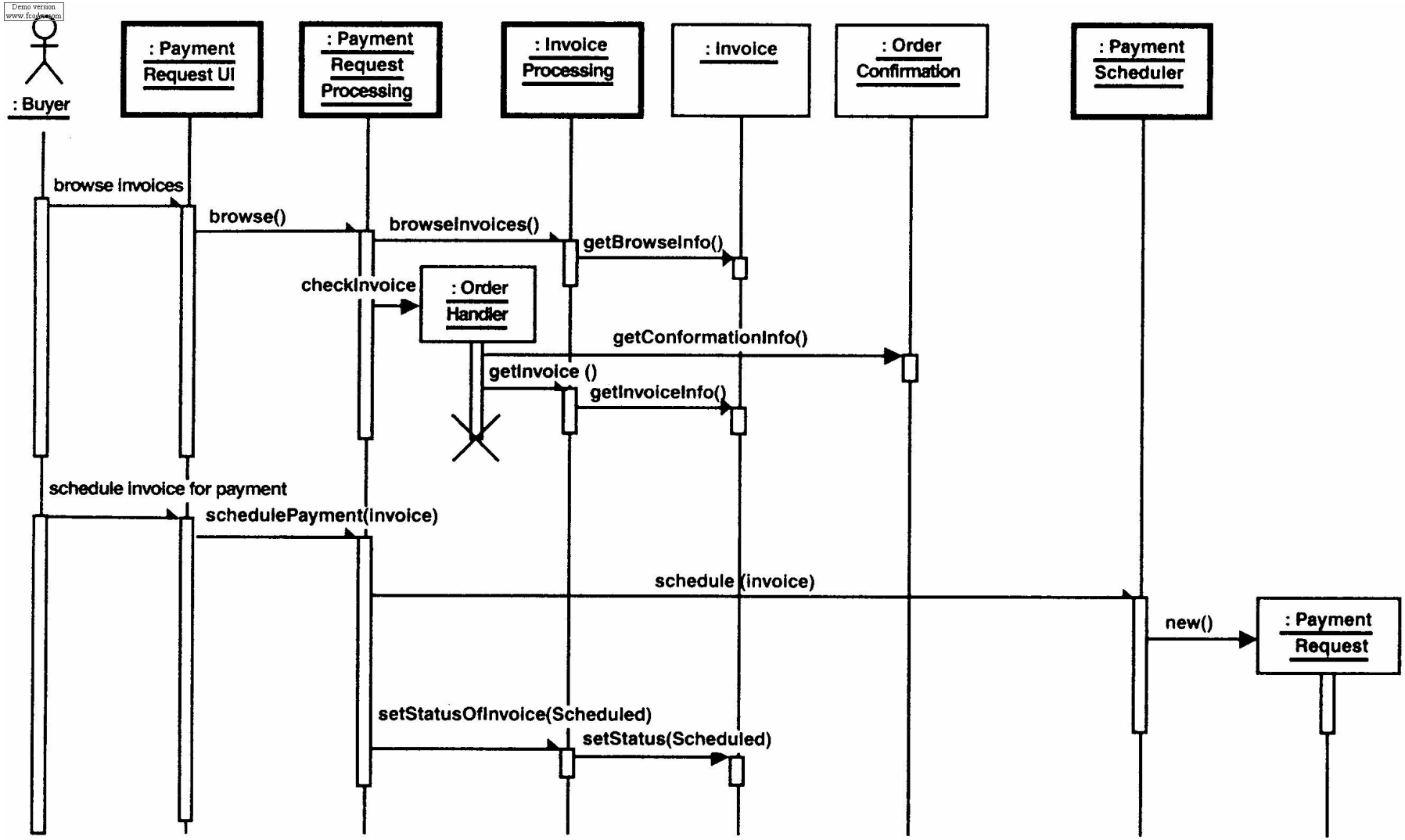
Design a Use Case

- Identify participating design classes
- Describe design object interactions
- Identify participating subsystems and interfaces
- Describe subsystem interactions
- Capture implementation requirements

Participating Design Classes



Design Object Interactions



Design a Class

- Outline the design class
  - see guidelines for boundary, entity, control classes
- Identify operations
- Identify attributes
- Identify associations and aggregations
- Identify generalizations
- Describe methods
- Describe states (if required)

Design a Subsystem

- Ensure subsystem is independent of other subsystems/interfaces
- Ensure subsystem provides right interface
- Ensure subsystem offers correct realization of the operations provided in its interface

Summary

- Design can be the trickiest part to get right
- How much is enough is less clear than
  - requirements analysis
  - implementation
- Architectural issues are more important than the details
- At least consider design tradeoffs